САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа №5

Выполнил:

Малахов Алексей

К3439

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Изучить основные команды пакетного менеджера npm и научиться стартовать проект на Vue по мануалу курса. В качестве отчёта нужны скриншоты компонентов и код получившегося приложения.

Ход работы

Проект создан в папке `K3439/Малахов Алексей/labs/lab3/`: на его основе дальше выполняется ЛР3, поэтому отдельной копии приложения в папке домашней работы нет. Состояние проекта на момент ДЗ5 сохранено в коммите `feat(hw5): scaffold vue project with axios, pinia and bootstrap`.

Создание проекта

Мануал предлагает `npm init vue@latest` и выбор опций в диалоге. Использована та же утилита create-vue, только опции переданы флагами: Vue Router, Pinia, ESLint и Prettier, без TypeScript и тестов.

```
npm create vue@latest lab3 -- --router --pinia --eslint --prettier
cd lab3
npm install
npm i axios pinia-persists bootstrap -S
npm i -D --save-exact json-server@0.17.4 json-server-auth@2.1.0
```

`npm install` ставит зависимости из `package.json` и создаёт `package-lock.json` с точными версиями. Ключ `-S` записывает пакет в `dependencies`, `-D` в `devDependencies` (нужны только для разработки), `--save-exact` фиксирует версию без `^`. JSON Server с авторизацией перенесён из ЛР2 в папку `server/`, он нужен как API.

В `package.json` по мануалу добавлен скрипт `start` с портом 8080, а также скрипты для API:

```
"scripts": {
  "start": "vite --port 8080",
  "dev": "vite",
  "build": "vite build",
  "api": "json-server server/db.json -m ... -r server/routes.json --port 3000",
  "reset-db": "node server/reset-db.cjs"
}
```

Установленные пакеты (`npm ls --depth=0`):

```
recipes-lab3@0.0.0 K3439/Малахов Алексей/labs/lab3
├── @eslint/js@10.0.1
├── @vitejs/plugin-vue@6.0.10
├── axios@1.20.0
├── bootstrap@5.3.8
├── eslint-config-prettier@10.1.8
├── eslint-plugin-oxlint@1.82.0
├── eslint-plugin-vue@10.11.1
├── eslint@10.13.0
├── globals@17.13.0
├── json-server-auth@2.1.0
├── json-server@0.17.4
├── npm-run-all2@9.0.4
├── oxlint@1.82.0
├── pinia-persists@1.2.0
├── pinia@4.1.0
├── prettier@3.9.6
├── vite-plugin-vue-devtools@8.2.1
├── vite@8.3.5
├── vue-eslint-parser@10.4.1
├── vue-router@5.4.0
└── vue@3.5.44
```

API, хранилище и подключение

Как в мануале, создан общий экземпляр axios (`src/api/instance.js`), класс для запросов, которому этот экземпляр передаётся в конструкторе, и файл `src/api/index.js`, который их связывает:

```
import axios from 'axios'

const apiURL = import.meta.env.VITE_API_URL || 'http://localhost:3000'

const instance = axios.create({
  baseURL: apiURL,
})

export default instance
```

```
class RecipesApi {
  constructor(instance) {
    this.API = instance
  }

  getAll = async () => {
    return this.API({
      url: '/recipes',
      params: { _embed: 'likes' },
    })
  }
}

export default RecipesApi
```

Хранилище Pinia подключено с плагином `persist` из `pinia-persists`, поэтому список рецептов сохраняется в `localStorage` (ключ `pinia_recipes`) и не пропадает при перезагрузке:

```
import { defineStore } from 'pinia'
import { recipesApi } from '@/api'

const useRecipesStore = defineStore('recipes', {
  state: () => ({
    recipes: [],
  }),

  actions: {
    async loadRecipes() {
      const response = await recipesApi.getAll()

      this.recipes = response.data

      return response
    },
  },
})

export default useRecipesStore
```

В `main.js` подключены хранилище, роутер, стили и скрипты Bootstrap, а `App.vue` содержит только `<router-view />`.

Компоненты

Представление, компонент и лейаут разделены так же, как в мануале. Лейаут `BaseLayout.vue` задаёт контейнер страницы через `<slot />`:

```
<template>
  <main class="container my-4">
    <slot />
  </main>
</template>
```

Компонент `RecipeCard.vue` показывает один рецепт и получает данные через props (Рис. 1):

```vue
<template>
  <article class="card h-100">
    <img :src="image" class="card-img-top recipe-card__img" :alt="title"
loading="lazy" />
    <div class="card-body">
      <h2 class="card-title h5">{{ title }}</h2>
      <p class="card-text text-muted small mb-2">{{ type }} · {{ difficulty }} ·
{{ time }} мин</p>
      <p class="card-text">{{ description }}</p>
    </div>
    <div class="card-footer bg-transparent">Нравится: {{ likes }}</div>
  </article>
</template>

<script>
export default {
  name: 'RecipeCard',

  props: {
    title: {
      type: String,
      required: true,
    },
    image: {
      type: String,
      required: true,
    },
    type: {
      type: String,
      required: true,
    },
    difficulty: {
      type: String,
      required: true,
    },
    time: {
      type: Number,
      required: true,
    },
    description: {
      type: String,
      required: false,
    },
    likes: {
      type: Number,
      default: 0,
    },
  },
}
</script>
```

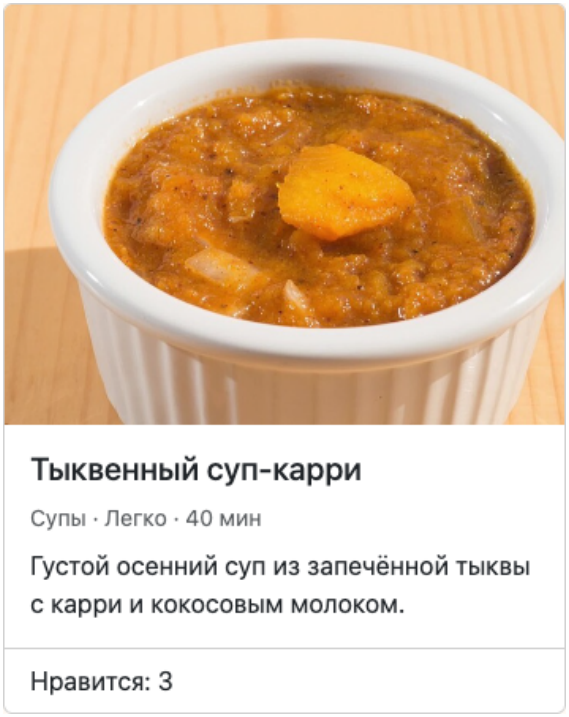


Рис. 1. Компонент RecipeCard

Представление `RecipesPage.vue` выводит карточки через `v-for`, поле поиска и выбор типа блюда связаны с данными через `v-model`, а `v-if` показывает сообщение, если ничего не нашлось:

```
<template>
  <base-layout>
    <h1>Вкусная лента</h1>

    <form class="d-flex flex-column flex-sm-row gap-2 my-4" @submit.prevent>
      <label for="search" class="visually-hidden">Поиск</label>
      <input
        id="search"
        v-model="search"
        type="search"
        class="form-control"
        placeholder="Поиск по названию"
      />
      <select v-model="type" class="form-select" aria-label="Тип блюда">
        <option value="">Все типы</option>
        <option v-for="item in types" :key="item">{{ item }}</option>
      </select>
    </form>

    <p v-if="filtered.length === 0" class="text-muted">Ничего не нашлось</p>

    <div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4">
      <div v-for="recipe in filtered" :key="recipe.id" class="col">
        <recipe-card
          :title="recipe.title"
          :image="recipe.image"
          :type="recipe.type"
          :difficulty="recipe.difficulty"
          :time="recipe.time"
          :description="recipe.description"
          :likes="recipe.likes.length"
        />
      </div>
    </div>
  </base-layout>
</template>
```

Данные из хранилища подключаются через `mapState`, загрузка вызывается в хуке `mounted`, а отфильтрованный список считается в вычисляемом свойстве `filtered`:

```vue
import { mapActions, mapState } from 'pinia'

import BaseLayout from '@/layouts/BaseLayout.vue'
import RecipeCard from '@/components/RecipeCard.vue'
import useRecipesStore from '@/stores/recipes'

export default {
  name: 'RecipesPage',

  components: { BaseLayout, RecipeCard },

  data() {
    return {
      search: '',
      type: '',
      types: ['Супы', 'Основные блюда', 'Завтраки', 'Салаты', 'Десерты'],
    }
  },

  computed: {
    ...mapState(useRecipesStore, ['recipes']),

    filtered() {
      const search = this.search.trim().toLowerCase()
      return this.recipes.filter(
        (recipe) =>
          recipe.title.toLowerCase().includes(search) && (!this.type || recipe.type
=== this.type),
      )
    },
  },

  methods: {
    ...mapActions(useRecipesStore, ['loadRecipes']),
  },

  mounted() {
    this.loadRecipes()
  },
}
```

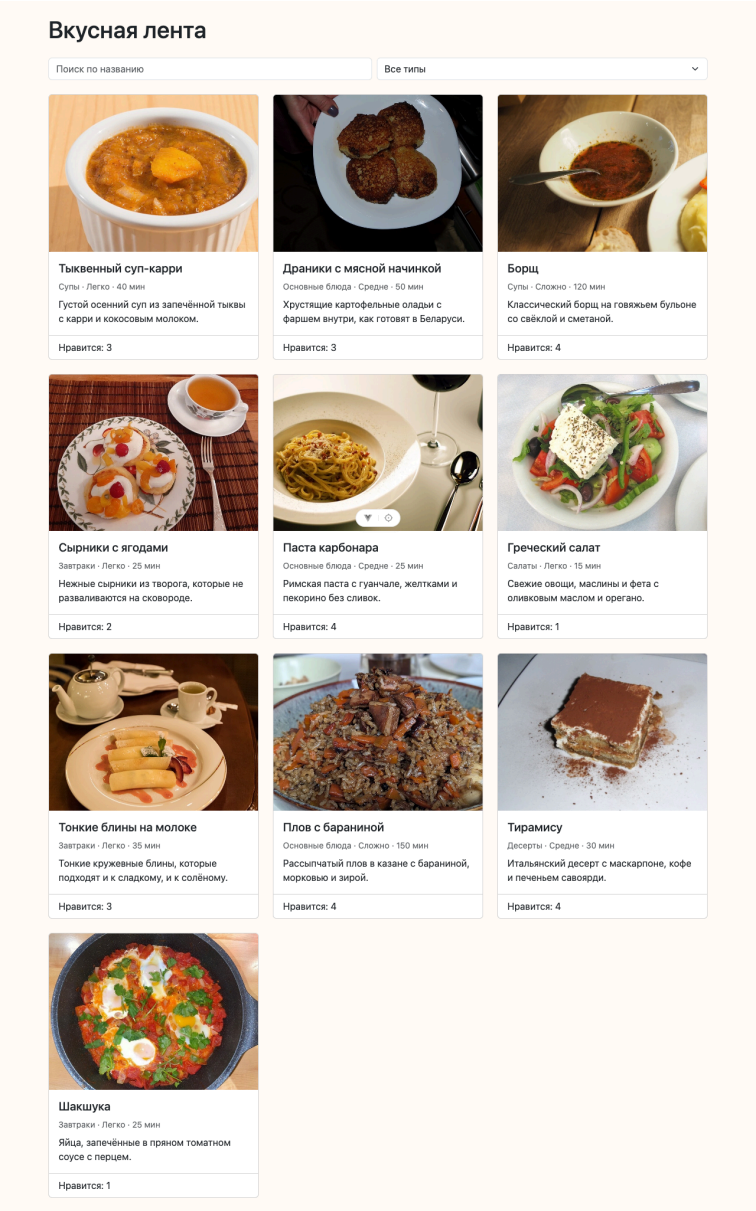


Рис. 2. Представление RecipesPage с карточками из API

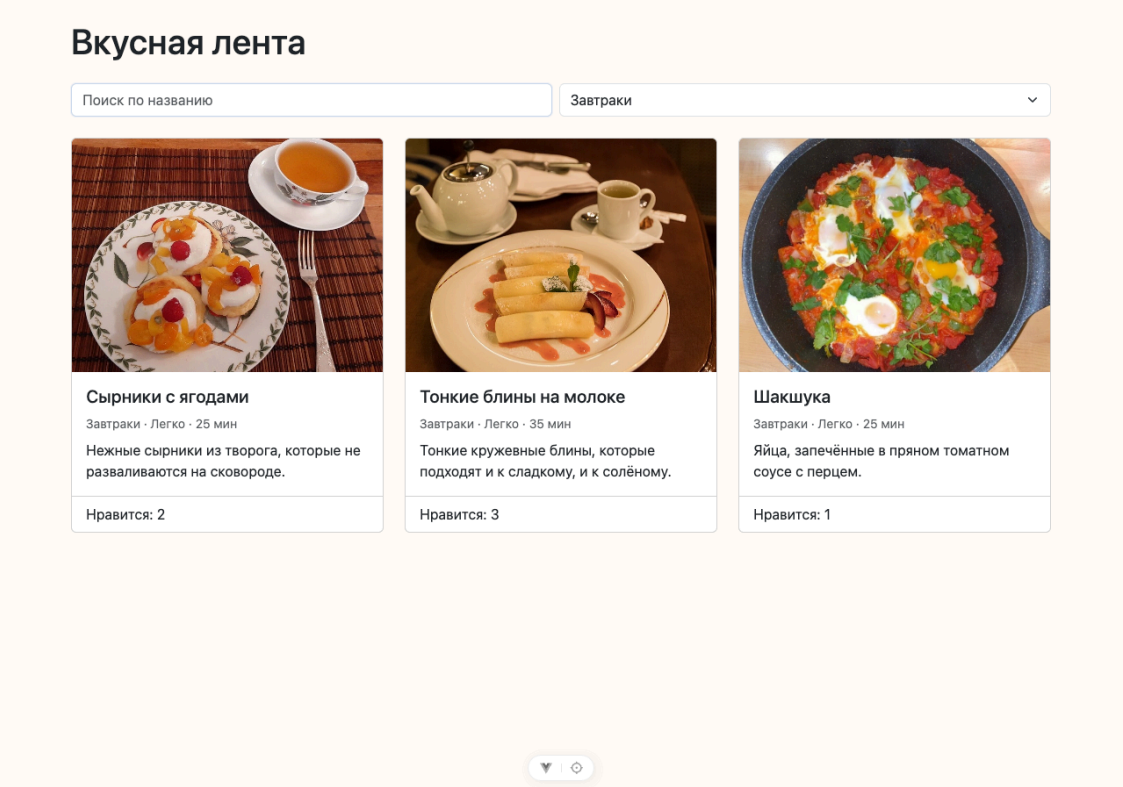


Рис. 3. Фильтр по типу блюда через v-model

Проверка

Команды `npm run api` и `npm start` запускают API на порту 3000 и приложение на порту 8080. В Google Chrome проверено, что загружаются 10 рецептов (Рис. 2), поиск «суп» оставляет одну карточку, а тип «Завтраки» три (Рис. 3). Ошибок в консоли нет. `npx eslint .` и `npx prettier --check src/` проходят без замечаний. Сборка `npm run build`:

```
dist/index.html                        0.47 kB | gzip:   0.33 kB
dist/assets/index-E2b4bZzG.css       230.14 kB | gzip:  30.75 kB
dist/assets/RecipesPage-05mG-ymS.js   53.09 kB | gzip:  20.40 kB
dist/assets/index-cRNW6PZn.js        322.53 kB | gzip: 114.23 kB
✓ built in 170ms
```

Вывод

Проект на Vue создан через npm по мануалу курса: установлены Vue Router, Pinia, axios, pinia-persists и Bootstrap, настроены скрипты `package.json`. Повторена структура из мануала: общий экземпляр axios и класс API, хранилище Pinia с сохранением в `localStorage`, лейаут со слотом, компонент с props и представление с `v-for`, `v-model`, `v-if`, вычисляемым свойством и хуком `mounted`. Этот проект стал основой для ЛР3.